GROUPE 1

TERAN GODOY Jesus
WABOE Hyacinthe
RANDRIANARISON Tsinjo
THEOPHILE Adrien
RASAMOELINA Ny Toavimpahendrena
EDERY Simon

UNIVERSITÉ TOULOUSE CAPITOLE
FACULTÉ D'INFORMATIQUE

Projet : Agence de voyages DIRECTION SOLEIL



Client: Direction Soleil

RAVAT Franck
BOULIC-BOUADJIO Audren
TOURNIER Ronan

Sommaire

Table des figures **4**
1 - Approche Logique **5**
1.1. Schéma Relationnel 5
1.2. Commentaires 6
2 - Requêtes SQL **7**
2.1. Requêtes SQL Demandées 7
2.1.1. Requête 1 7
2.1.2. Requête 2 8
2.1.3. Requête 3 9
2.1.4. Requête 4 10
2.1.5. Requête 5 12
2.1.6. Requête 6 13
2.1.7. Requête 7 13
2.1.8. Requête 8 14
2.1.9. Requête 9 16
2.1.10. Requête 10 18
2.2. Requêtes SQL Supplémentaires 19
2.2.1. Chiffre d'affaires total par départ 19
2.2.2. Arbitrage de prix entre présélections d'un même circuit 21
2.2.3. Animations accessibles pendant les séjours 23
2.2.4. Départs proposés sans réservation 24
2.2.5. Départs prévus pendant les mois les plus chauds 26
2.3. Requêtes d'extraction 28
2.3.1. Requête d'extraction - TABLE_PRINCIPALE 28
2.3.2. Requête d'extraction - TABLE_ZONES 29
2.3.3. Requête d'extraction - TABLE_DEPARTS 30
3 - Transformation des données et rapports PowerBI **31**
3.1. Transformation des données 31
3.2. Rapports PowerBI 32
3.2.1. Synthèse direction: 32
3.2.2. Planning et Remplissage 33
3.2.3. Destination et Climat 34
4 - Annexes **36**
4.1. Création des tables de la base de données 36
4.1.1. Table CLIENTS 36
4.1.2. Table AEROPORTS_DEP 36
4.1.3. Table CIRCUITS 36
4.1.4. Table CHOIX 37
4.1.5. Table RESERVER 37
4.1.6. Table MAJORATION 38
4.1.7. Table HOTELS 38

4.1.8. Table LOGER..... 39
4.1.9. Table ANIMATIONS.....39
4.1.10. Table ACTIVITE.....39
4.1.11. Table VALIDITE..... 40
4.1.12. Table REALISER..... 40
4.1.13. Table SITES..... 40
4.1.14. Table VISITER.....41
4.1.15. Table MOIS.....41
4.1.16. Table PAYS.....42
4.1.17. Table AVIS.....42
4.1.18. Table Zone_Geo.....42
4.1.19. Table SITUER.....43
4.1.20. Table GUIDES..... 43
4.1.21. Table ACCOMPAGNER.....43
4.1.22. Table LANGUE..... 44
4.1.23. Table PRATIQUER..... 44
4.2. Création de la vue ANIMATIONS_H..... 44
4.3 Création du trigger sur RESERVER.....45

Table des figures

Figure 1 : Résultat de la requête 1 sur SQL Developer.....7
Figure 2 : Résultat de la requête 2 sur SQL Developer.....8
Figure 3 : Résultat de la requête 3 sur SQL Developer.....9
Figure 4 : Résultat de la requête 4 sur SQL Developer.....11
Figure 5 : Résultat de la requête 5 sur SQL Developer.....12
Figure 6 : Résultat de la requête 6 sur SQL Developer.....13
Figure 7 : Résultat de la requête 7 sur SQL Developer.....14
Figure 8: Résultat de la requête 8 sur SQL Developer.....15
Figure 9: Résultat de la requête 9 sur SQL Developer.....17
Figure 10: Résultat de la requête 10 sur SQL Developer.....18
Figure 11: Résultat de la requête supplémentaire 1 sur SQL Developer..... 20
Figure 12: Résultat de la requête supplémentaire 2 sur SQL Developer.....22
Figure 13: Résultat de la requête supplémentaire 3 sur SQL Developer.....23
Figure 14: Résultat de la requête supplémentaire 4 sur SQL Developer.....25
Figure 15: Résultat de la requête supplémentaire 5 sur SQL Developer.....27
Figure 16: Tableau de bord Power BI sur la partie synthèse direction:..... 32
Figure 17 : Tableau de bord PowerBI sur la planification des circuits et le remplissage..... 33
Figure 18 : Tableau de bord Power BI sur la partie destinations et climat.....34

1 - Approche Logique

1.1. Schéma Relationnel

A partir du modèle conceptuel de données proposé par le client, nous avons élaboré le modèle logique de données ci-dessous :

CLIENTS(CodeC, NomC, PrénomC, TelC, MelC)

AEROPORTS_DEP(CodeA, NomA)

CIRCUITS(CodeCI, NomCI, PrixBase,NbPersMax, #CodeCI_Est_Complete)

PAYS(CodeP, NomP, AvisGen)

SITES(CodeS, NomS, TypeS, DescripS)

ANIMATIONS(CodeAN, NomAN, TypeAN, HeureDeb, JourSemaine, #CodeH)

HOTELS(CodeH, NomH, Categorie, VilleH, TelH, MelH)

GUIDES(CodeG, NomG, PrenomG, DatePremier)

ZONE_GEO(CodeZ, ZoneGeo)

ACTIVITE(CodeAC, NomAC, TarifAC)

LANGUE(CodeL, NomL)

MOIS(CodeMois, LibelleMois)

CHOIX (#CodeC, #CodeA, #CodeCI, DateDepartC, OrdrePreference)

RESERVER (#CodeC, #CodeA, #CodeCI, DateDepartR, DateResa, NbPers)

MAJORATION (#CodeA, #CodeCI, DateDepartM, MontantMaj)

PRATIQUER (#CodeG, #CodeL, NiveauL)

VALIDITE (#CodeAC, DateDebut, DateFin)

REALISER (#CodeCI, #CodeAC, DateDepartRe, JourSem)

LOGER (#CodeCI, #CodeH, JourSe)

VISITER (#CodeCI, #CodeS, JourSe2, HeureVisite, DureeVisite)

AVIS (#CodeP, #CodeMois, TempMin, TempMax, AvisMens, TempMinMoy, TempMaxMoy)

SITUER (#CodeCI, #CodeZ, #CodeP)

ACCOMPAGNER (#CodeG, #CodeCI, DateDepartA)

1.2. Commentaires

Lors de l'élaboration du modèle relationnel, on s'est aperçu de quelques modifications indispensables pour garantir le bon fonctionnement de la base de données :

- Un changement des noms des attributs "Date" donnés sur le dictionnaire de données était nécessaire. Pour éviter toute ambiguïté avec le type de données DATE sous Oracle, nous avons renommé les attributs concernés. Donc pour la relation RESERVER par exemple, au lieu de mettre "Date" comme nom, nous avons plutôt défini "DateDepartR". Les noms d'attributs choisis sont : *DateDepartC*, *DateDepartM*, *DateDepartA*, *DateDepartRe* pour les relations suivantes respectivement : CHOIX, MAJORATION, ACCOMPAGNER et REALISER.
- Tous les noms des relations ont été définis en majuscules.
- Tous les attributs ou tables ayant des accents ont été écrits sans accents.

2 - Requêtes SQL

2.1. Requêtes SQL Demandées

2.1.1. Requête 1

Énoncé : Donner le montant total du circuit "Les saveurs tunisiennes", partant de l'aéroport de Toulouse le 08/07/2026, réservé par Monsieur Medhi TERRANER.

Requête SQL :

```
SELECT cl.nomC, cl.prenomC, r.dateDepartR, c.nomCI, ad.nomA, c.prixBase +
m.montantMaj AS montant_total_du_circuit
FROM Circuits c, Majoration m, Aeroports_Dep ad, Reserver r, Clients cl
WHERE c.codeCI = m.codeCI
AND m.codeA = ad.codeA
AND ad.codeA = r.codeA
AND r.codeC = cl.codeC
AND r.codeCI = c.codeCI
AND r.dateDepartR = m.dateDepartM
AND LOWER(c.nomCI) = 'les saveurs tunisiennes'
AND LOWER (ad.nomA) = 'toulouse'
AND r.dateDepartR = TO_DATE('08/07/2026', 'DD/MM/YYYY')
AND LOWER (cl.prenomC) = 'medhi'
AND LOWER (cl.nomC) = 'terraner';
```

Résultat :

	NOMC	PRENOMC	DATEDEPARTR	NOMCI	NOMA	MONTANT_TOTAL_DU_CIRCUIT
1	TERRANER	Medhi	08/07/26	Les saveurs tunisiennes	Toulouse	780

Figure 1 : Résultat de la requête 1 sur SQL Developer

Justification :

La requête calcule le montant du circuit en additionnant son prix de base (c.prixBase) et la majoration (m.montantMaj), qui dépend de l'aéroport et de la date de départ. Elle sélectionne le circuit “Les saveurs tunisiennes”, réservé par Medhi TERRANER au départ de Toulouse le 08/07/2026.

Pour vérifier le résultat, on constate dans la table CIRCUITS que le prix de base est de 700 €. Dans MAJORATION, le départ concerné possède une majoration de 80 €. On obtient donc 700 + 80 = 780 €, ce qui correspond bien au montant retourné par la requête.

2.1.2. Requête 2

Énoncé : Donner la liste des circuits marocains permettant de séjourner à la fois dans l'hôtel "LION D'OR" et l'hôtel "EDEN".

Requête SQL :

```
SELECT  DISTINCT c.CodeCI, c.NomCI
FROM circuits c, Situer s, Pays p
WHERE c.CodeCI = s.CodeCI
AND s.CodeP = p.CodeP
AND UPPER(p.NomP) = 'MAROC'
AND c.codeCI IN (
   SELECT I.codeci
   FROM Hotels h, loger I
   WHERE h.codeH = I.codeH
   AND UPPER(h.NomH) = 'LION D''OR'
)
AND c.codeCI IN (
   SELECT I2.codeci
   FROM Hotels h2, loger I2
   WHERE h2.codeH = I2.codeH
   AND UPPER(h2.NomH) = 'EDEN'
);
```

Résultat :

	CODECI	NOMCI
1	8	Marrakech et Atlas
2	6	Médinas impériales

Figure 2 : Résultat de la requête 2 sur SQL Developer

Justification :

La requête sélectionne les circuits situés au Maroc à l'aide des tables SITUER et PAYS. Elle utilise ensuite deux sous-requêtes sur LOGER et HOTELS pour vérifier que chaque circuit permet de séjourner à la fois dans les hôtels "Lion d'Or" et "Eden".

Pour vérifier le résultat, on constate dans la table HOTELS que ces deux hôtels correspondent aux codes 1 et 2. Dans LOGER, les circuits 6 et 8 sont associés à ces deux hôtels. La table SITUER confirme qu'ils se déroulent au Maroc. On retrouve donc "Médinas impériales" et "Marrakech et Atlas", ce qui correspond bien aux deux circuits retournés par la requête.

2.1.3. Requête 3

Énoncé : Donner pour chaque pays le mois le plus chaud (en tenant compte des valeurs moyennes).

Requête SQL :

```
SELECT P.NomP, M.LibelleMois, A.TempMaxMoy
FROM PAYS P, MOIS M, AVIS A
WHERE P.CodeP = A.CodeP
AND M.CodeMois = A.CodeMois
AND A.TempMaxMoy = (
   SELECT MAX(A2.TempMaxMoy)
   FROM AVIS A2
   WHERE A2.CodeP = P.CodeP
);
```

Résultat :

	NOMP	LIBELLEMOIS	TEMPMAXMOY
1	Tunisie	Août	35
2	Maroc	Juillet	38
3	Grèce	Juillet	34
4	Italie	Août	32
5	Espagne	Août	30
6	Turquie	Août	32
7	Égypte	Juillet	37
8	Croatie	Juillet	31

Figure 3 : Résultat de la requête 3 sur SQL Developer

Justification :

On relie les tables PAYS, MOIS et AVIS pour récupérer le nom du pays, le mois et sa température moyenne. La sous-requête avec MAX(A2.TempMaxMoy) cherche ensuite la température la plus élevée propre à chaque pays, ce qui permet de filtrer la liste et de ne garder que le mois le plus chaud.

2.1.4. Requête 4

Énoncé : Donner pour chaque départ de chaque circuit le nombre de réservations et le nombre de places disponibles.

Requête SQL : Nous considérons uniquement les départs pour lesquels au moins une réservation a été effectuée.

```
SELECT c.NomCI, a.NomA, r.DateDepartR,
COUNT(*) AS NbReservations,
c.NbPersMax -  (
        SELECT SUM(r2.NbPers)
        FROM RESERVER r2
        WHERE r2.CodeCI = c.CodeCI
        AND r2.DateDepartR = r.DateDepartR
    ) AS PlacesDisponibles

FROM CIRCUITS c, AEROPORTS_DEP a, RESERVER r
WHERE c.CodeCI = r.CodeCI
AND a.CodeA = r.CodeA
GROUP BY c.CodeCI, c.NomCI, c.NbPersMax,
a.CodeA, a.NomA, r.DateDepartR
ORDER BY c.NomCI, a.NomA, r.DateDepartR;
```

Résultat :

	NOMCI	NOMA	DATEDEPARTR	NBRESERVATIONS	PLACESDISPONIBLES
1	Andalousie authentique	Biarritz	01/07/26	2	7
2	Andalousie authentique	Bordeaux	01/07/26	4	7
3	Andalousie authentique	Marseille	18/02/26	5	15
4	Andalousie authentique	Montpellier	08/07/26	5	12
5	Andalousie authentique	Paris CDG	07/11/26	3	22
6	Andalousie authentique	Paris CDG	19/12/26	3	21
7	Andalousie authentique	Paris CDG	25/09/27	5	18
8	Andalousie authentique	Toulouse	26/03/25	3	23
9	Andalousie authentique	Toulouse	07/05/25	9	0
10	Andalousie authentique	Toulouse	01/07/26	3	7
11	Catalogne et Costa Brava	Bordeaux	30/08/25	8	11
12	Catalogne et Costa Brava	Bordeaux	10/04/27	6	12
13	Catalogne et Costa Brava	Marseille	02/04/25	1	24
14	Catalogne et Costa Brava	Marseille	11/06/25	4	20
15	Catalogne et Costa Brava	Marseille	11/04/26	8	8

...

177	Tunisie antique et médinas	Carcassonne	08/07/26	1	10
178	Tunisie antique et médinas	Lyon	18/01/25	3	23
179	Tunisie antique et médinas	Lyon	01/07/26	2	14
180	Tunisie antique et médinas	Lyon	08/07/26	3	10
181	Tunisie antique et médinas	Paris Orly	27/03/27	4	14
182	Tunisie antique et médinas	Toulouse	10/06/26	6	13
183	Tunisie antique et médinas	Toulouse	01/07/26	2	14
184	Tunisie antique et médinas	Toulouse	08/07/26	4	10
185	Valence et Costa Blanca	Bordeaux	30/04/25	3	16
186	Valence et Costa Blanca	Bordeaux	02/07/25	9	5
187	Valence et Costa Blanca	Bordeaux	26/07/25	6	6
188	Valence et Costa Blanca	Bordeaux	20/09/25	6	9
189	Valence et Costa Blanca	Bordeaux	08/07/26	6	7
190	Valence et Costa Blanca	Bordeaux	22/08/26	5	11
191	Valence et Costa Blanca	Brest	08/07/26	1	7
192	Valence et Costa Blanca	Limoges	08/07/26	1	7

Figure 4 : Résultat de la requête 4 sur SQL Developer

Justification :

La requête permet de calculer le nombre de réservations et de places disponibles pour chaque départ ayant au moins une réservation. Elle regroupe les réservations par circuit, aéroport et date de départ, puis utilise COUNT(*) pour compter les réservations. Le nombre de places disponibles est obtenu en soustrayant à la capacité maximale du circuit (c.NbPersMax) le nombre total de

voyageurs inscrits à la même date, tous aéroports confondus, grâce à une sous-requête sur RESERVER.

Pour vérifier les résultats, on constate dans la table RESERVER que le circuit "Les saveurs tunisiennes", au départ de Toulouse le 08/07/2026, possède 5 réservations pour un total de 14 personnes. Cependant, ce même circuit compte 7 autres voyageurs depuis les autres aéroports à cette date, soit 21 personnes au total. Dans CIRCUITS, sa capacité maximale est de 30 personnes. On obtient donc 30 - 21 = 9 places disponibles, ce qui correspond bien au résultat retourné par la requête pour ce départ.

2.1.5. Requête 5

Énoncé : Donner le nom et le prénom des guides ayant effectué le plus de circuits différents.

Requête SQL :

```
SELECT g.nomG, g.prenomG
FROM guides g, accompagner a
WHERE g.codeG = a.codeG
GROUP BY g.codeG, g.nomG, g.prenomG
HAVING COUNT(DISTINCT a.codeCI ) >= ALL (
        SELECT COUNT(DISTINCT a2.codeCI)
        FROM accompagner a2
        GROUP BY a2.codeG );
```

Résultat :

	NOMG	PRENOMG
1	BENNANI	Youssef
2	HASSAN	Omar

Figure 5 : Résultat de la requête 5 sur SQL Developer

Justification :

Pour chaque guide ayant accompagné au moins un circuit, on va compter le nombre de circuits différents qu'ils ont effectués HAVING COUNT(DISTINCT a.codeCI). On va comparer ce résultat avec le nombre de circuits calculé pour tous les autres guides dans la sous-requête, grâce à l'opérateur >= ALL.

2.1.6. Requête 6

Énoncé : Donner le nombre de clients ayant réservé un voyage en 2025 sans avoir fait au préalable une sélection de préférence.

Requête SQL :

```
SELECT COUNT( DISTINCT c.codec) as "Nombre de clients"
FROM clients c, reserver r
WHERE c.codec = r.codec
AND TO_CHAR( r.DateResa, 'yyyy') = '2025'
AND c.codec NOT IN (
  SELECT ch.codec
  FROM choix ch
  WHERE r.DateDepartR = ch.DateDepartC
  AND r.codeA = ch.codeA
  AND r.codeCI = ch.codeCI
);
```

Résultat :

	Nombre de clients
1	233

Figure 6 : Résultat de la requête 6 sur SQL Developer

Justification :

La requête compte les clients distincts ayant effectué une réservation en 2025 sans présélection correspondante dans la table CHOIX. La condition NOT IN permet d'exclure les clients ayant présélectionné le même circuit, le même aéroport et la même date de départ.

Cependant, la table CHOIX ne contient pas de date d'enregistrement des préférences. Il n'est donc pas possible de vérifier précisément si une présélection a été effectuée avant la réservation.

2.1.7. Requête 7

Énoncé : Donner les coordonnées des guides ayant assuré tous les circuits du Maroc cette année.

Requête SQL : les coordonnées complètes des guides ne sont pas disponibles dans notre modèle, on ne peut donc citer que le nom et prénom

```
SELECT g.nomg, g.prenomg
FROM guides g
```

```
WHERE NOT EXISTS (
   SELECT 1
   FROM pays p, situer s
   WHERE p.codeP = s.codeP
   AND LOWER(p.nomp) = 'maroc'
   AND NOT EXISTS (
      SELECT 1
      FROM circuits c, accompagner a
      WHERE c.codeCI = a.codeCI
      AND c.codeCI = s.codeCI
      AND a.codeG = g.codeG
      AND TO_CHAR( a.dateDepartA, 'yyyy') = TO_CHAR(SYSDATE, 'yyyy') ));
```

Résultat :

	NOMG	PRENOMG
1	BENNANI	Youssef

Figure 7 : Résultat de la requête 7 sur SQL Developer

Justification :

La requête permet d'identifier les guides ayant assuré tous les circuits du Maroc durant l'année 2026. Elle utilise les tables GUIDES, PAYS, SITUER, CIRCUITS et ACCOMPAGNER. Les deux sous-requêtes avec NOT EXISTS permettent de vérifier qu'il n'existe aucun circuit marocain que le guide n'a pas accompagné durant l'année en cours.

Pour vérifier le résultat, on constate dans SITUER que le Maroc possède 5 circuits, correspondant aux codes 6, 7, 8, 9 et 10. Dans ACCOMPAGNER, on retrouve le guide BENNANI Youssef associé à chacun de ces circuits avec des dates de départ en 2026. Aucun autre guide n'a assuré l'ensemble de ces 5 circuits. On retrouve donc uniquement BENNANI Youssef, ce qui correspond au résultat attendu.

2.1.8. Requête 8

Énoncé : Donner les coordonnées du client ayant effectué le plus de circuits pour chaque pays.

Requête SQL :

```
SELECT p.nomP, COUNT(DISTINCT r.codeci) AS nbCircuits, c.nomc, c.prenomc,
c.telc, c.melc
FROM clients c, reserver r, circuits ci, situer s, pays p
WHERE c.codec = r.codec
```

```
AND r.codeci = ci.codeci
AND ci.codeci = s.codeci
AND s.codep = p.codep
AND r.dateDepartR <= SYSDATE
GROUP BY c.codec, c.nomc, c.prenomc, c.telc, c.melc, p.codep, p.nomP
HAVING COUNT(DISTINCT r.codeci)  =
   (
      SELECT MAX(COUNT(DISTINCT r2.codeci))
      FROM reserver r2, circuits ci2, situer s2, pays p2
      WHERE r2.codeci = ci2.codeci
      AND ci2.codeci = s2.codeci
      AND s2.codep = p2.codep
      AND p.codep = p2.codep
      AND r2.dateDepartR <= SYSDATE
      GROUP BY r2.codeC
   )
ORDER BY p.nomP ASC;
```

Résultat :

	NOMP	NBCIRCUITS	NOMC	PRENOMC	TELC	MELC
1	Croatie	1	LAURENT	Emma	+33635303332	laurent.emma@outlook.fr
2	Croatie	1	GAUTHIER	Samuel	+33663719384	s.gauthier.1334@example.org
3	Croatie	1	RENARD	Elise	+33730387486	renard.elise@icloud.com
4	Croatie	1	PICARD	Samuel	+33667652410	picard.samuel@gmail.com
5	Croatie	1	BESSON	Tom	+33774050817	besson.tom@outlook.fr
6	Croatie	1	REY	Etienne	+33684598564	rey.etienne@yahoo.fr
7	Croatie	1	DELMAS	Aurelie	+33683038115	delmas.aurelie@outlook.fr
8	Croatie	1	ROLLAND	Salome	+33655861271	rolland.salome@yahoo.fr
9	Croatie	1	LEMOINE	Ophelie	+33728111847	lemoine.ophelie@gmail.com
10	Croatie	1	ROUX	Agathe	+33689499950	roux.agathe@gmail.com
11	Croatie	1	SANCHEZ	Ambre	+33639549688	sanchez.ambre@icloud.com
12	Croatie	1	NICOLAS	Alice	+33681594442	alice.nicolas.113@example.org
13	Croatie	1	LAURENT	Alice	+33697491233	alice.laurent.129@example.org
14	Croatie	1	MATHIEU	Amine	+33639630441	amine.mathieu.136@example.org
15	Croatie	1	ROUSSEL	Hector	+33610924582	hector.roussel.166@example.org
...						
111	Turquie	1	BARBIER	Elsa	+33620655178	elsa.barbier.1063@example.org
112	Turquie	1	DORVAL	Léa	+33645475333	lea.dorval.1087@example.org
113	Turquie	1	CORREIA	Baptiste	+33669615408	b.correia.1099@example.org
114	Turquie	1	BESSON	Alya	+33655446066	alya.besson.1178@example.org
115	Turquie	1	DESCHAMPS	Sébastien	+33687047730	s.deschamps.1179@example.org
116	Turquie	1	HADDAD	Elise	+33635617364	elise.haddad.1214@example.org
117	Turquie	1	MICHEL	Amine	+33662488283	amine.michel.1215@example.org
118	Turquie	1	MASSON	Florian	+33661183521	f.masson.1245@example.org
119	Turquie	1	LEMAIRE	Léo	+33672218864	leo.lemaire.1248@example.org
120	Turquie	1	ROUSSEAU	Mehdi	+33666765920	m.rousseau.1249@example.org
121	Turquie	1	BOYER	Florian	+33624634344	florian.boyer.1305@example.org
122	Turquie	1	BERNARD	Sophie	+33616324373	bernard.sophie@gmail.com
123	Turquie	1	BOYER	Kevin	+33637125296	boyer.kevin@gmail.com

Figure 8: Résultat de la requête 8 sur SQL Developer

Justification :

La requête permet d'identifier les clients ayant effectué le plus de circuits différents dans chaque pays. Elle utilise les tables CLIENTS, RESERVER, CIRCUITS, SITUER et PAYS pour associer les clients aux pays visités. La condition r.DateDepartR <= SYSDATE permet de ne prendre en compte que les circuits déjà effectués. Ensuite, COUNT(DISTINCT r.CodeCI) compte les circuits différents de chaque client, et la sous-requête permet de sélectionner le nombre maximal de circuits pour chaque pays.

Pour vérifier les résultats, on compare les réservations dans RESERVER avec les pays associés aux circuits dans SITUER. On retrouve notamment “Medhi TERRANER” avec 2 circuits en Tunisie, “Marie DUPONT” avec 2 circuits au Maroc, etc… . En Espagne, “Julie ROUSSEAU” et “Océane MATHIEU” sont à égalité avec 2 circuits chacune. Pour la Turquie, l'Égypte et la Croatie, le maximum est de 1 circuit, ce qui explique la présence de plusieurs clients à égalité. Ces valeurs correspondent bien aux résultats retournés par la requête.

2.1.9. Requête 9

Énoncé : Donner les noms des circuits disponibles au départ de Toulouse qui permettent de visiter tous les sites de Tunisie.

Requête SQL :

```
SELECT DISTINCT c.NomCI
FROM circuits c, majoration m, aeroports_dep a, situer s, pays p
WHERE c.CodeCI = m.CodeCI
AND m.CodeA = a.CodeA
AND c.CodeCI = s.CodeCI
AND s.CodeP = p.CodeP
AND LOWER(a.NomA) = 'toulouse'
AND LOWER(p.NomP) = 'tunisie'
AND m.datedepartm NOT IN (
  SELECT r.dateDepartR
  FROM reserver r
  WHERE r.CodeCI = c.CodeCI
  GROUP BY r.dateDepartR
  HAVING SUM(r.NbPers) >= c.NbPersMax
)
AND NOT EXISTS (
  SELECT 1
  FROM situer s2, visiter v
  WHERE s2.CodeCI = v.CodeCI
  AND s2.CodeP = p.CodeP
```

```
  AND NOT EXISTS (
    SELECT 1
    FROM visiter v2
    WHERE v2.CodeCI = c.CodeCI
    AND v2.CodeS = v.CodeS
  )
);
```

Résultat :

	NOMCI
1	Perles de Tunisie

Figure 9: Résultat de la requête 9 sur SQL Developer

Justification :

La requête permet d'identifier les circuits proposés au départ de Toulouse qui visitent tous les sites touristiques associés à la Tunisie et disposent encore de places. Nous utilisons une double négation avec NOT EXISTS pour vérifier qu'il n'existe aucun site tunisien que le circuit ne visite pas.

La condition NOT IN permet d'exclure les dates où le circuit est complet, en tenant compte des réservations de tous les aéroports. Dans notre modèle, les sites ne sont pas directement associés à un pays. Nous identifions donc les sites tunisiens à partir des sites visités par les circuits situés en Tunisie.

Cette méthode fonctionne avec notre jeu de données, mais suppose que l'ensemble de ces visites couvre tous les sites touristiques tunisiens concernés.

2.1.10. Requête 10

Énoncé : Donner le nom du pays proposant plus de circuits que la Grèce.

Requête SQL :

```
SELECT P.nomp
FROM Pays P, Situer S
WHERE P.codep = S.codep
GROUP BY P.codep, P.nomp
HAVING COUNT( DISTINCT S.codeci) > (
   SELECT COUNT( DISTINCT S.codeci)
   FROM Pays P, Situer S
   WHERE P.codep = S.codep
   AND LOWER(P.nomp) = 'grèce');
```

Résultat :

	NOMP
1	Tunisie
2	Maroc

Figure 10: Résultat de la requête 10 sur SQL Developer

Justification :

La requête permet d'identifier les pays proposant plus de circuits que la Grèce. Elle utilise les tables PAYS et SITUER pour associer chaque pays à ses circuits. La fonction COUNT(DISTINCT S.CodeCI) permet de compter les circuits différents de chaque pays, puis la clause HAVING compare ce nombre à celui de la Grèce grâce à une sous-requête.

Pour vérifier les résultats, on constate dans la table SITUER que la Grèce possède 3 circuits différents, tandis que la Tunisie et le Maroc en possèdent 5 chacun. Les autres pays proposent au maximum 3 circuits différents. On retrouve donc uniquement la Tunisie et le Maroc, ce qui correspond bien aux deux pays retournés par la requête.

2.2. Requêtes SQL Supplémentaires

2.2.1. Chiffre d'affaires total par départ

Énoncé : Pour chaque départ ayant au moins une réservation, donner le nom du circuit, l'aéroport, la date, le nombre total de voyageurs inscrits et le chiffre d'affaires total généré. Le chiffre d'affaires est calculé à partir du nombre de personnes réservées, multiplié par la somme du prix de base et de la majoration. Classer les résultats par chiffre d'affaires décroissant.

Utilité métier : Comparer les départs selon leur chiffre d'affaires afin d'identifier ceux qui génèrent le plus de revenus pour l'agence.

Requête SQL :

```
SELECT CI.NomCI, A.NomA, R.DateDepartR, SUM(R.NbPers) AS
TotalVoyageurs,
    SUM(R.NbPers * (CI.PrixBase + M.MontantMaj)) AS ChiffreAffaires
FROM CIRCUITS CI, AEROPORTS_DEP A, MAJORATION M, Reserver R
WHERE CI.CodeCI = R.CodeCI
AND A.CodeA = R.CodeA
AND M.CodeCI = R.CodeCI
AND M.CodeA = R.CodeA
AND M.DateDepartM = R.DateDepartR
GROUP BY CI.CodeCI, CI.NomCI, A.CodeA, A.NomA, R.DateDepartR
ORDER BY ChiffreAffaires DESC;
```

Résultat :

	NOMCI	NOMA	DATEDEPARTR	TOTALVOYAGEURS	CHIFFREAFFAIRES
1	Rome, Florence et Venise	Lyon	18/10/25	32	32640
2	Les délices égéens (Grece)	Toulouse	15/08/26	30	31020
3	Météores et Delphes	Clermont-Ferrand	28/06/25	26	24960
4	Andalousie authentique	Toulouse	07/05/25	30	24330
5	Rome, Florence et Venise	Nice	19/07/25	22	22704
6	Les saveurs tunisiennes	Toulouse	21/10/26	30	22440
7	Les saveurs tunisiennes	Lyon	30/05/26	30	22230

…

184	Côtes dalmates	Metz	07/08/27	3	2649
185	Médinas impériales	Perpignan	01/07/26	3	2565
186	Catalogne et Costa Brava	Marseille	08/07/26	3	2184
187	Perles de Tunisie	Poitiers	01/07/26	3	2175
188	Les saveurs tunisiennes	Paris CDG	08/07/26	3	2160
189	Côtes dalmates	Marseille	15/07/26	2	1696
190	Tunisie antique et médinas	Carcassonne	08/07/26	2	1596
191	Valence et Costa Blanca	Limoges	08/07/26	1	817
192	Médinas impériales	Paris CDG	01/05/26	1	805

Figure 11: Résultat de la requête supplémentaire 1 sur SQL Developer

Justification :

La requête permet de calculer le nombre total de voyageurs et le chiffre d'affaires généré pour chaque départ. Elle relie les tables CIRCUITS, AEROPORTS_DEP, MAJORATION et RESERVER afin d'associer chaque réservation à son circuit, son aéroport et sa date de départ. La clause GROUP BY regroupe les données par départ unique (circuit, aéroport et date).

Pour chaque groupe, SUM(R.NbPers) calcule le nombre des voyageurs, tandis que SUM(R.NbPers * (CI.PrixBase + M.MontantMaj)) calcule le chiffre d'affaires total du depart. Enfin, ORDER BY ChiffreAffaires DESC classe les résultats pour faire ressortir les départs générant le plus de chiffre d'affaires en premier.

Pour vérifier les résultats, on constate dans RESERVER que le circuit “Rome, Florence et Venise”, au départ de Lyon le 18/10/2025, possède 11 réservations pour un total de 32 voyageurs. Dans CIRCUITS, son prix de base est de 980 €, auquel s'ajoute une majoration de 40 € dans MAJORATION. On obtient donc 32 × (980 + 40) = 32 640 €, ce qui correspond au chiffre d'affaires calculé pour ce départ.

2.2.2. Arbitrage de prix entre présélections d'un même circuit

Énoncé : Pour chaque client ayant présélectionné le même circuit avec plusieurs dates ou aéroports de départ, afficher son nom, son prénom, le circuit, l'aéroport, la date, l'ordre de préférence et le prix total de chaque présélection (prix de base + majoration). Trier les résultats par client et par prix décroissant.

Utilité métier : Observer comment un client classe des départs d'un même circuit dont les prix diffèrent, afin de mieux comprendre l'influence des tarifs sur ses préférences.

Requête SQL :

```
SELECT C.CodeC, C.NomC, C.PrenomC, CI.NomCI,
       A.NomA, CH.DateDepartC, CH.OrdrePreference,
       CI.PrixBase + M.MontantMaj AS PrixTotal
FROM CLIENTS C, CIRCUITS CI, CHOIX CH,
     AEROPORTS_DEP A, MAJORATION M
WHERE C.CodeC = CH.CodeC
AND CI.CodeCI = CH.CodeCI
AND A.CodeA = CH.CodeA
AND M.CodeCI = CH.CodeCI
AND M.CodeA = CH.CodeA
AND M.DateDepartM = CH.DateDepartC
AND EXISTS (
    SELECT *
    FROM CHOIX CH2
    WHERE CH2.CodeC = CH.CodeC
    AND CH2.CodeCI = CH.CodeCI
    AND (CH2.DateDepartC <> CH.DateDepartC
    OR CH2.CodeA <> CH.CodeA)
)
ORDER BY C.CodeC, CI.NomCI, PrixTotal DESC;
```

Résultat :

	CODEC	NOMC	PRENOMC	NOMCI	NOMA	DATEDEPARTC	ORDREPREFERENCE	PRIXTOTAL
1	13	TERRANER	Medhi	Les saveurs tunisiennes	Toulouse	08/07/26	1	780
2	13	TERRANER	Medhi	Les saveurs tunisiennes	Toulouse	01/07/26	2	745
3	18	BLANC	Maxime	Les saveurs tunisiennes	Paris CDG	08/07/26	2	720
4	18	BLANC	Maxime	Les saveurs tunisiennes	Paris CDG	01/07/26	1	720
5	35	VIDAL	Mathieu	Les délices égéens (Turquie)	Toulouse	01/07/26	2	985
6	35	VIDAL	Mathieu	Les délices égéens (Turquie)	Toulouse	24/06/26	1	966
7	84	FLEURY	Mailys	Les délices égéens (Turquie)	Toulouse	08/07/26	3	988
8	84	FLEURY	Mailys	Les délices égéens (Turquie)	Toulouse	01/07/26	2	985
9	136	MATHIEU	Amine	Côtes dalmates	Marseille	08/08/26	1	862
10	136	MATHIEU	Amine	Côtes dalmates	Paris CDG	01/09/26	2	838
11	235	LECLERC	Adrien	Les délices égéens (Turquie)	Bordeaux	17/07/27	1	974
12	235	LECLERC	Adrien	Les délices égéens (Turquie)	Bordeaux	11/09/27	2	968
13	241	MATHIEU	Alice	Côtes dalmates	Nantes	19/06/27	1	898
14	241	MATHIEU	Alice	Côtes dalmates	Nice	31/07/27	2	872

…

32	1636	LEROUX	Ines	Côtes dalmates	Nice	31/07/27	2	872
33	1662	CORREIA	Tom	Les saveurs tunisiennes	Bordeaux	24/02/27	1	773
34	1662	CORREIA	Tom	Les saveurs tunisiennes	Marseille	15/05/27	2	738
35	1716	PONS	Bilal	Du Caire à Assouan	Toulouse	09/10/27	2	1077
36	1716	PONS	Bilal	Du Caire à Assouan	Toulon	17/04/27	1	1038
37	1720	BOYER	Sarah	Catalogne et Costa Brava	Marseille	14/08/27	1	745
38	1720	BOYER	Sarah	Catalogne et Costa Brava	Bordeaux	10/04/27	2	717
39	1741	RENAUD	Bastien	Les délices égéens (Turquie)	Bordeaux	11/09/27	2	968
40	1741	RENAUD	Bastien	Les délices égéens (Turquie)	Paris CDG	21/11/26	1	966
41	1759	NOEL	Asma	Andalousie authentique	Paris CDG	19/12/26	1	776
42	1759	NOEL	Asma	Andalousie authentique	Paris CDG	25/09/27	2	765

Figure 12: Résultat de la requête supplémentaire 2 sur SQL Developer

Justification :

La requête permet d'identifier les clients ayant présélectionné un même circuit avec des dates ou des aéroports de départ différents. Elle utilise les tables CLIENTS, CIRCUITS, CHOIX, AEROPORTS_DEP et MAJORATION. La sous-requête avec EXISTS permet de vérifier qu'un client possède plusieurs présélections pour un même circuit. Le prix de chaque présélection est ensuite calculé en additionnant le prix de base (CI.PrixBase) et la majoration (M.MontantMaj).

Pour vérifier les résultats, on constate dans CHOIX que Medhi TERRANER a présélectionné le circuit "Les saveurs tunisiennes" au départ de Toulouse pour deux dates différentes. Dans CIRCUITS, son prix de base est de 700 €. Dans MAJORATION, on retrouve 80 € pour le 08/07/2026 et 45 € pour le 01/07/2026. On obtient donc 780 € pour son premier choix et 745 € pour son deuxième choix. En effectuant cette vérification sur les autres présélections, on retrouve 42 lignes concernant 21 clients, ce qui correspond aux données de notre base.

Proposition d'ajout de données :

Nous pourrions ajouter dans la table CHOIX une troisième présélection pour “Medhi TERRANER” concernant le circuit “Les saveurs tunisiennes”, avec une autre date ou un autre aéroport de départ déjà présent dans MAJORATION. Cela permettrait de vérifier que la requête affiche bien les différentes possibilités choisies par un même client et compare leurs prix en fonction des majorations.

2.2.3. Animations accessibles pendant les séjours

Énoncé : Donner, pour chaque circuit, le nom des hôtels proposant une animation le jour où les voyageurs y séjournent, en précisant le jour, le nom de l'animation et son heure de début.

Utilité métier : Permettre à l'agence de présenter aux clients les animations correspondant réellement aux jours de séjour prévus dans les hôtels.

Requête SQL :

```
SELECT DISTINCT CI.NomCI, H.NomH, L.JourSe, A.NomAN,
    TO_CHAR(A.HeureDeb, 'HH24:MI') AS Heure
FROM CIRCUITS CI, LOGER L, HOTELS H, ANIMATIONS A
WHERE CI.CodeCI = L.CodeCI
AND L.CodeH = H.CodeH
AND H.CodeH = A.CodeH
AND L.JourSe = A.JourSemaine
ORDER BY CI.NomCI, L.JourSe;
```

Résultat :

	NOMCI	NOMH	JOURSE	NOMAN	HEURE
1	Andalousie authentique	Sevilla Plaza	Lundi	Danse Flamenco Traditionnelle	20:30
2	Catalogne et Costa Brava	Costa Brava	Mercredi	Danse Andalouse	20:45
3	Les délices égéens (Grece)	Rhodes Palace	Jeudi	Valse et Danses de Salon	18:30
4	Les délices égéens (Turquie)	Bosphorus Grand	Lundi	Cabaret Transformiste	21:30
5	Les délices égéens (Turquie)	Bodrum Marine	Vendredi	Jazz au Bord de la Mer	19:00
6	Sahara et oasis tunisiens	Djerba Golf	Jeudi	Karaoké Duo et Groupe	20:30

Figure 13: Résultat de la requête supplémentaire 3 sur SQL Developer

Justification :

La requête permet d'identifier les animations proposées dans les hôtels pendant les jours où les voyageurs y séjournent. Elle utilise les tables CIRCUITS, LOGER, HOTELS et ANIMATIONS pour associer chaque circuit à ses hôtels, puis compare L.JourSe et A.JourSemaine afin de retenir uniquement les animations programmées le jour du séjour.

Pour vérifier les résultats, on constate dans LOGER que le circuit "Andalousie authentique" prévoit un séjour à l'hôtel "Sevilla Plaza" le lundi. Dans ANIMATIONS, cet hôtel propose justement une animation "Danse Flamenco Traditionnelle" le lundi à 20 h 30. En effectuant cette vérification pour les autres circuits, on retrouve 6 animations correspondant aux jours de séjour, réparties sur 5 circuits, ce qui correspond aux données de notre base.

2.2.4. Départs proposés sans réservation

Énoncé : Donner les circuits, les aéroports et les dates de départ proposés par l'agence pour lesquels aucune réservation n'a été effectuée, en précisant le prix total du voyage (prix de base + majoration).

Utilité métier : Repérer les départs qui n'ont attiré aucun client, afin d'étudier l'intérêt de modifier les prix ou les dates proposés.

Requête SQL :

```
SELECT CI.NomCI, A.NomA, M.DateDepartM,
    CI.PrixBase + M.MontantMaj AS PrixTotal
FROM CIRCUITS CI, AEROPORTS_DEP A, MAJORATION M
WHERE CI.CodeCI = M.CodeCI
AND A.CodeA = M.CodeA
AND NOT EXISTS (
  SELECT *
  FROM RESERVER R
  WHERE R.CodeCI = M.CodeCI
  AND R.CodeA = M.CodeA
  AND R.DateDepartR = M.DateDepartM
)
ORDER BY CI.NomCI, M.DateDepartM;
```

Résultat :

	NOMCI	NOMA	DATEDEPARTM	PRIXTOTAL
1	Andalousie authentique	Toulouse	08/03/25	812
2	Andalousie authentique	Paris CDG	09/12/26	771
3	Désert de Merzouga	Marseille	15/07/26	915
4	Du Caire à Assouan	Paris CDG	24/11/27	1021
5	Du Caire à Assouan	Paris CDG	22/12/27	1023
6	Les Cyclades en liberté	Lyon	01/07/26	1119
7	Les délices égéens (Grece)	Toulouse	01/07/26	1010
8	Les délices égéens (Turquie)	Toulouse	12/07/25	996
9	Les délices égéens (Turquie)	Paris CDG	24/06/26	960
10	Les délices égéens (Turquie)	Toulouse	08/07/26	988
11	Marrakech et Atlas	Toulouse	01/07/26	745
12	Marrakech et Atlas	Pau	22/08/26	761
13	Perles de Tunisie	Toulouse	15/05/26	685
14	Perles de Tunisie	Marseille	01/07/26	716
15	Perles de Tunisie	Paris CDG	09/12/26	703
16	Rome, Florence et Venise	Paris CDG	08/07/26	1005
17	Sahara et oasis tunisiens	Toulouse	01/07/26	880
18	Sahara et oasis tunisiens	Toulouse	08/07/26	885
19	Sahara et oasis tunisiens	Toulouse	13/11/27	892
20	Sahara et oasis tunisiens	Toulouse	29/12/27	897
21	Sicile et volcans	Marseille	18/11/26	891

Figure 14: Résultat de la requête supplémentaire 4 sur SQL Developer

Justification :

La requête permet d'identifier les départs proposés par l'agence pour lesquels aucune réservation n'a été effectuée. Elle utilise les tables CIRCUITS, AEROPORTS_DEP et MAJORATION pour retrouver les départs et calculer leur prix total (CI.PrixBase + M.MontantMaj). La sous-requête avec NOT EXISTS permet ensuite d'exclure les départs présents dans RESERVER, en comparant le circuit, l'aéroport et la date de départ.

Pour vérifier les résultats, on constate dans MAJORATION que le circuit « Perles de Tunisie » propose un départ de Toulouse le 15/05/2026, avec un prix de base de 650 € et une majoration de 35 €, soit 685 €. Dans RESERVER, aucune réservation ne correspond à cette offre. En effectuant cette vérification sur tous les départs proposés, on retrouve 21 offres sans réservation, ce qui correspond aux résultats obtenus avec nos données.

Proposition d'ajout de données :

Nous pourrions ajouter dans la table MAJORATION un nouveau départ pour un circuit et un aéroport existants, sans ajouter de réservation correspondante dans

RESERVER. Cela permettrait de vérifier que la requête retrouve bien ce nouveau départ parmi ceux qui n'ont aucune réservation.

2.2.5. Départs prévus pendant les mois les plus chauds

Énoncé : Donner le nom des circuits, le pays, l'aéroport et la date de départ pour lesquels la température maximale moyenne du mois de départ est supérieure ou égale à 35 °C, en précisant cette température.

Utilité métier : Identifier les départs organisés pendant les périodes de fortes chaleurs pour mieux informer les clients et, si nécessaire, adapter les activités du programme.

Requête SQL :

```
SELECT DISTINCT CI.NomCI, P.NomP,
      A.NomA, M.DateDepartM, AV.TempMaxMoy
FROM CIRCUITS CI, SITUER S, PAYS P, AVIS AV,
    MAJORATION M, AEROPORTS_DEP A
WHERE CI.CodeCI = S.CodeCI
AND S.CodeP = P.CodeP
AND P.CodeP = AV.CodeP
AND CI.CodeCI = M.CodeCI
AND M.CodeA = A.CodeA
AND AV.CodeMois = EXTRACT(MONTH FROM M.DateDepartM)
AND AV.TempMaxMoy >= 35
ORDER BY AV.TempMaxMoy DESC, CI.NomCI;
```

Résultat :

	NOMCI	NOMP	NOMA	DATEDEPARTM	TEMPMAXMOY
1	Désert de Merzouga	Maroc	Marseille	15/07/26	38
2	Marrakech et Atlas	Maroc	Toulouse	08/07/26	38
3	Marrakech et Atlas	Maroc	Bordeaux	01/07/26	38
4	Marrakech et Atlas	Maroc	Toulouse	01/07/26	38
5	Marrakech et Atlas	Maroc	Pau	08/07/26	38
6	Médinas impériales	Maroc	Marseille	01/07/26	38
7	Médinas impériales	Maroc	Marseille	08/07/26	38
8	Médinas impériales	Maroc	Toulouse	08/07/26	38
9	Médinas impériales	Maroc	Toulouse	01/07/26	38
10	Médinas impériales	Maroc	Perpignan	01/07/26	38
11	Rif et Chefchaouen	Maroc	Nîmes	01/07/26	38
12	Rif et Chefchaouen	Maroc	Paris CDG	08/07/26	38
13	Route des kasbahs	Maroc	Lyon	01/07/26	38
14	Marrakech et Atlas	Maroc	Pau	22/08/26	37
15	Djerba, détente et culture	Tunisie	Toulouse	15/08/26	35
16	Les saveurs tunisiennes	Tunisie	Paris CDG	22/08/26	35

Figure 15: Résultat de la requête supplémentaire 5 sur SQL Developer

Justification :

La requête permet d'identifier les départs prévus pendant les mois où la température maximale moyenne est supérieure ou égale à 35 °C. Elle utilise les tables CIRCUITS, SITUER et PAYS pour identifier le pays de chaque circuit, puis MAJORATION pour récupérer les dates de départ. La fonction EXTRACT(MONTH FROM M.DateDepartM) permet de retrouver le mois correspondant dans AVIS. La condition AV.TempMaxMoy >= 35 permet ensuite de sélectionner uniquement les départs concernés.

Pour vérifier les résultats, on constate dans MAJORATION que le circuit "Médinas impériales" propose un départ de Toulouse le 01/07/2026. La table SITUER indique que ce circuit se déroule au Maroc et, dans AVIS, la température maximale moyenne du mois de juillet est de 38 °C. Ce départ respecte donc bien la condition demandée. En effectuant cette vérification sur les autres départs, on retrouve 16 offres concernées, réparties sur 7 circuits différents dans notre jeu de données.

2.3. Requêtes d'extraction

Nos requêtes d'extraction recensent les données sources nécessaires pour les sujets d'analyse dans la base de données.

Nous avons choisi de créer plusieurs requêtes d'extraction afin d'éviter les doublons dans nos données et pour que ces dernières soient cohérentes dans la représentation visuelle.

2.3.1. Requête d'extraction - TABLE_PRINCIPALE

Objectif : récupérer les informations commerciales des réservations pour calculer le chiffre d'affaires, le panier moyen, le nombre de voyageurs et les statistiques par pays et par aéroport.

```
CREATE TABLE TABLE_PRINCIPALE AS
SELECT c.CodeCI, c.NomCI,
      r.CodeC, r.CodeA, ad.NomA,
      r.DateDepartR AS DateDepartCI,
      r.DateResa AS DateReservationCI,
      r.NbPers,
      c.PrixBase, m.MontantMaj,
      p.CodeP, p.NomP,
      r.NbPers * (c.PrixBase + m.MontantMaj) AS CACI,
      c.NbPersMax,
      mo.CodeMois, mo.LibelleMois,
      av.TempMin, av.TempMax,
      av.TempMinMoy, av.TempMaxMoy, av.AvisMens,
      ac.DateDepartA, g.NomG, g.PrenomG
FROM CIRCUITS c, RESERVER r,
    AEROPORTS_DEP ad, MAJORATION m,
    AVIS av, MOIS mo, ACCOMPAGNER ac, GUIDES g,
    (SELECT DISTINCT s.CodeCI, p.CodeP, p.NomP
     FROM SITUER s, PAYS p
     WHERE s.CodeP = p.CodeP) p
WHERE c.CodeCI = r.CodeCI
AND r.CodeA = ad.CodeA
AND m.CodeCI = r.CodeCI
AND m.CodeA = r.CodeA
AND m.DateDepartM = r.DateDepartR
AND p.CodeCI = c.CodeCI
AND p.CodeP = av.CodeP
AND mo.CodeMois = av.CodeMois
```

```
AND mo.CodeMois = EXTRACT(MONTH FROM r.DateDepartR)
AND ac.DateDepartA = r.DateDepartR
AND c.CodeCI = ac.CodeCI
AND ac.CodeG = g.CodeG;
```

Résultats : 800 réservations, 2 163 voyageurs et 1 879 485 € de chiffre d'affaires.

2.3.2. Requête d'extraction - TABLE_ZONES

Objectif : répartir le chiffre d'affaires et les voyageurs entre les zones géographiques traversées par chaque circuit, sans compter plusieurs fois les montants dans le total général.

```
CREATE TABLE TABLE_ZONES AS
SELECT t.NomP, z.ZoneGeo,
      SUM(t.CACI / nz.NbZones) AS CA_Reparti,
      SUM(t.NbPers / nz.NbZones) AS Voyageurs_Repartis
FROM TABLE_PRINCIPALE t, SITUER s, ZONE_GEO z,
    (SELECT CodeCI, CodeP, COUNT(*) AS NbZones
     FROM SITUER
     GROUP BY CodeCI, CodeP) nz
WHERE t.CodeCI = s.CodeCI
AND t.CodeP = s.CodeP
AND s.CodeZ = z.CodeZ
AND s.CodeCI = nz.CodeCI
AND s.CodeP = nz.CodeP
GROUP BY t.NomP, z.ZoneGeo;
```

Résultats : 28 combinaisons pays/zone, avec un total de 1 879 485 € de CA réparti et 2 163 voyageurs équivalents.

A savoir :

Pour calculer le chiffre d'affaires par zone géographique, nous avons choisi de le répartir équitablement lorsqu'un circuit traverse plusieurs zones. Par exemple, si un circuit rapporte 10 000 € et traverse deux zones, chaque zone reçoit 5 000 €. Nous avons appliqué le même principe aux voyageurs. Cela peut donner des valeurs décimales, comme 2,5 voyageurs. Cette répartition reste théorique et permet d'éviter les doubles comptages.

2.3.3. Requête d'extraction - TABLE_DEPARTS

Objectif : récupérer les informations sur les départs, y compris ceux sans réservation, afin de calculer les taux de remplissage et les places disponibles dans Power BI.

```
CREATE TABLE TABLE_DEPARTS AS
SELECT m.CodeCI, c.NomCI,
      m.CodeA, a.NomA,
      m.DateDepartM AS DateDepart,
      c.NbPersMax,
      m.MontantMaj,
      COUNT(r.CodeC) AS NbReservations,
      SUM(r.NbPers) AS NbVoyageurs
FROM MAJORATION m
JOIN CIRCUITS c
  ON m.CodeCI = c.CodeCI
JOIN AEROPORTS_DEP a
  ON m.CodeA = a.CodeA
LEFT JOIN RESERVER r
  ON r.CodeCI = m.CodeCI
 AND r.CodeA = m.CodeA
 AND r.DateDepartR = m.DateDepartM
GROUP BY m.CodeCI, c.NomCI,
       m.CodeA, a.NomA,
       m.DateDepartM,
       c.NbPersMax,
       m.MontantMaj;
```

Résultat : 213 offres de départ, correspondant à 174 départs distincts (circuit + date), dont 21 offres sans réservation.

Importation de tables complémentaires dans Power BI :

En plus des trois tables d'extraction (TABLE_PRINCIPALE, TABLE_ZONES et TABLE_DEPARTS), nous avons également importé les tables GUIDES, ACCOMPAGNER, AVIS, MOIS et PAYS depuis Oracle.

Ces tables nous ont permis de compléter nos tableaux de bord, notamment pour suivre les affectations des guides par année et analyser les températures des différents pays selon les mois, en les comparant aux moyennes des dix dernières années.

3 - Transformation des données et rapports PowerBI

3.1. Transformation des données

Les données extraites doivent être préparées avant leur utilisation dans Power BI. Pour les départs sans réservation, la colonne NbVoyageurs peut contenir des valeurs nulles. Nous les avons donc remplacées par 0 dans Power Query pour faciliter les calculs et l'affichage des indicateurs.

1. Sur Power BI, ouvrir l'onglet "Transformer les données".

2. Sélectionner la colonne NbVoyageurs.

3. Cliquer sur "Remplacer les valeurs" : null → 0.

4. Vérifier que les départs sans réservation affichent bien 0 voyageur.

5. Cliquer sur "Fermer et appliquer".

3.2. Rapports PowerBI

3.2.1. Synthèse direction:

Ce premier tableau de bord permet d'avoir une vision globale de la santé de l'agence.

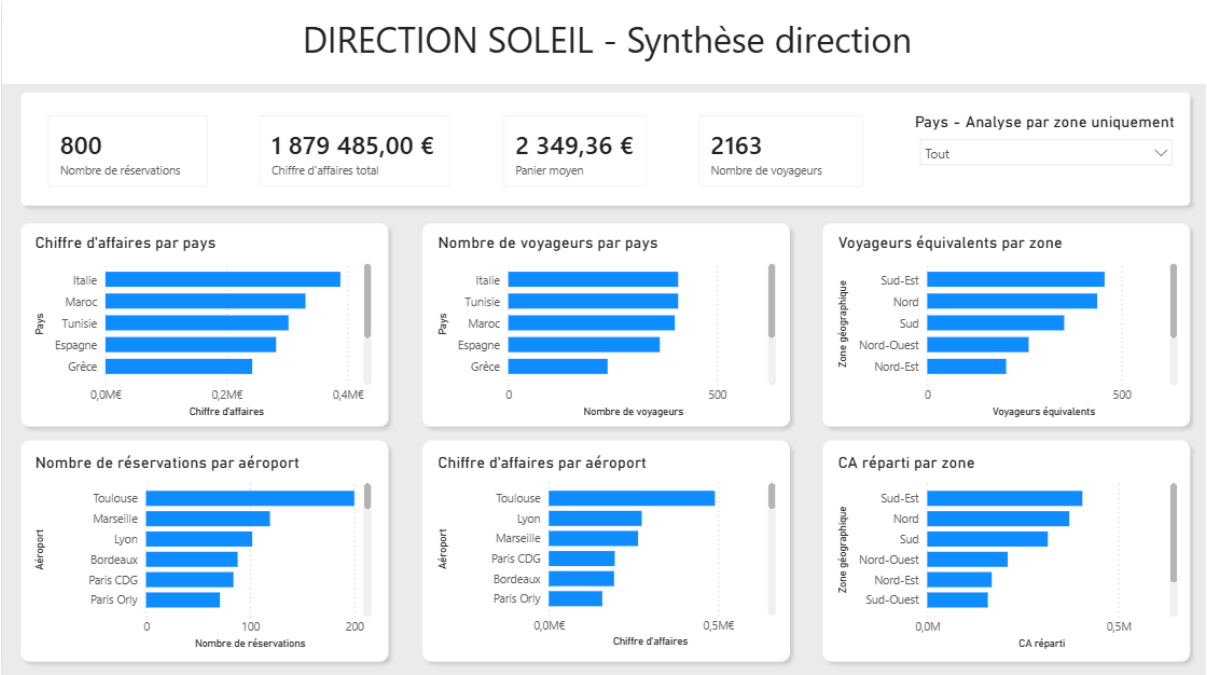


Figure 16: Tableau de bord Power BI sur la partie synthèse direction:

Ce tableau de bord présente le chiffre d'affaires total de l'agence ainsi que sa répartition par pays, par aéroport et par zone géographique, à partir des réservations des départs programmés entre 2025 et 2027.

Ces indicateurs permettent d'avoir un suivi financier de l'activité de l'agence sur différents axes (pays, zone géographique, aéroport) afin d'aider à orienter la prise de décision de l'agence sur les différents circuits à promouvoir.

Ensuite, on retrouve aussi des informations relatives aux clients de l'agence. Comme le *nombre de réservations* faites par les clients et le *nombre de voyageurs* pour les différents circuits proposés par l'agence.

Enfin, on a également la répartition du *nombre de voyageurs par pays*. Cet indicateur permet de connaître le succès de différents pays auprès des clients afin de cibler les différents pays pour l'implantation de nouveaux circuits.

3.2.2. Planning et Remplissage

L'analyse de l'historique permet d'identifier les circuits et les périodes qui se remplissent peu et d'y proposer des promotions, puis de suivre les départs à venir pour agir avant la date de départ.

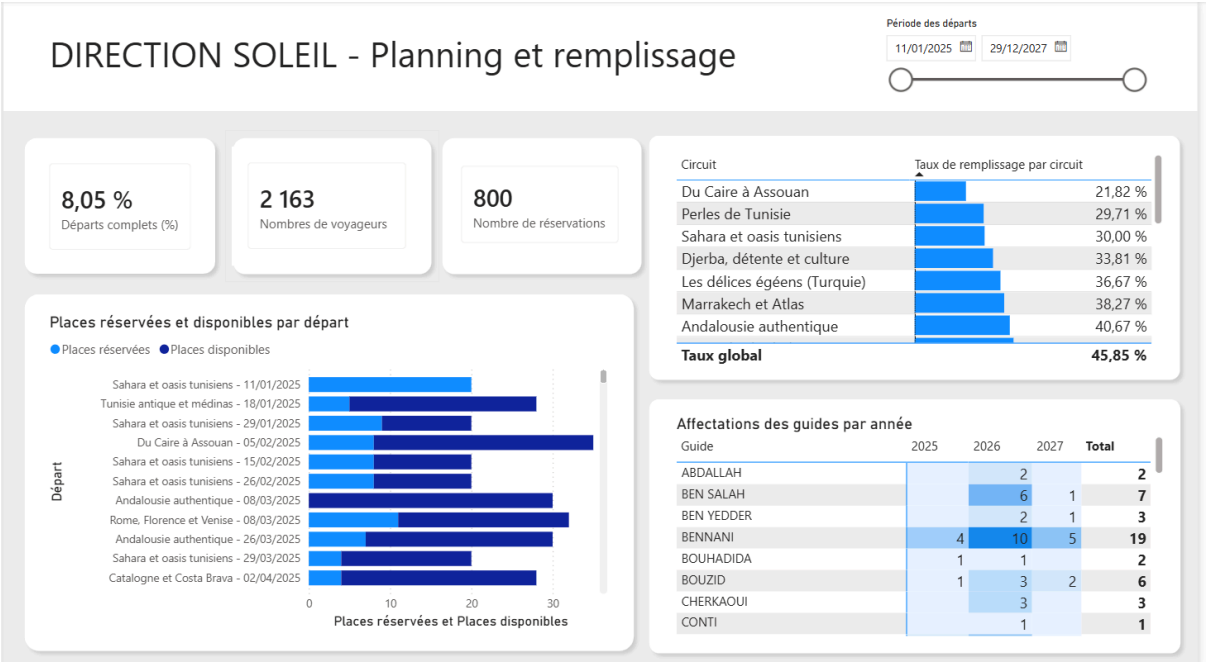


Figure 17 : Tableau de bord PowerBI sur la planification des circuits et le remplissage

Ce tableau de bord s'adresse à l'exploitation. Il répond à trois questions :

1. *Quels départs risquent de rester vides ?*
2. *Quels départs ou circuits n'ont pas été remplis par le passé ?*
3. *Comment les affectations sont-elles réparties entre les guides ?*

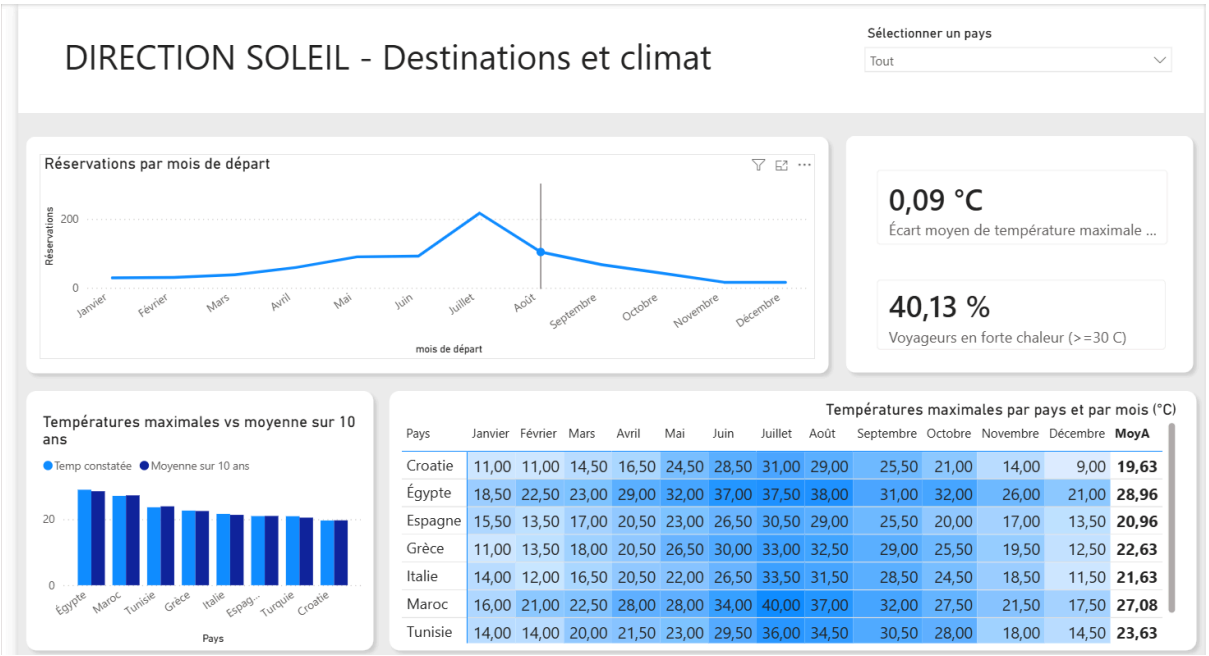
Le segment « Période des départs » permet de changer la période de façon interactive. Cela laissera donc la liberté au client de se placer sur les départs à venir ou sur une période passée.

Pour élaborer ce tableau de bord, on a choisi ces indicateurs pour les raisons suivantes:

- **Départs complets (%)** : part des départs dont toutes les places sont réservées. Une carte suffit pour cette valeur unique.
- **Nombre de voyageurs et de réservations** : volumes de base dont découlent les autres indicateurs, en cartes pour une lecture immédiate.
- **Places réservées et disponibles par départ** : repère les départs peu remplis, passés ou à venir.
- **Taux de remplissage par circuit** : compare les circuits entre eux. Les plus bas sont les candidats à une promotion.
- **Affectations des guides par année** : permet de comparer le nombre d'affectations de chaque guide selon les années et de mieux visualiser leur répartition.

3.2.3. Destination et Climat

Ce tableau de bord permet d'avoir une vision sur la saisonnalité afin d'ajuster la tarification et d'adapter le catalogue aux conditions météorologiques de chaque territoire. Il vise à optimiser la stratégie commerciale en identifiant précisément les destinations et les périodes de l'année à promouvoir en priorité.



Pays	Janvier	Février	Mars	Avril	Mai	Juin	Juillet	Août	Septembre	Octobre	Novembre	Décembre	MoyA
Croatie	11,00	11,00	14,50	16,50	24,50	28,50	31,00	29,00	25,50	21,00	14,00	9,00	**19,63**
Égypte	18,50	22,50	23,00	29,00	32,00	37,00	37,50	38,00	31,00	32,00	26,00	21,00	**28,96**
Espagne	15,50	13,50	17,00	20,50	23,00	26,50	30,50	29,00	25,50	20,00	17,00	13,50	**20,96**
Grèce	11,00	13,50	18,00	20,50	26,50	30,00	33,00	32,50	29,00	25,50	19,50	12,50	**22,63**
Italie	14,00	12,00	16,50	20,50	22,00	26,50	33,50	31,50	28,50	24,50	18,50	11,50	**21,63**
Maroc	16,00	21,00	22,50	28,00	28,00	34,00	40,00	37,00	32,00	27,50	21,50	17,50	**27,08**
Tunisie	14,00	14,00	20,00	21,50	23,00	29,50	36,00	34,50	30,50	28,00	18,00	14,50	**23,63**

Figure 18 : Tableau de bord Power BI sur la partie destinations et climat

Le tableau de bord permet d'isoler une destination en particulier ou de conserver une vision globale du catalogue de destinations.

Les deux indicateurs permettent de suivre la proportion de voyageurs ayant réservé un départ pendant un mois où la température maximale moyenne atteint au moins 30 °C, ainsi que l'écart moyen entre les températures maximales constatées et les moyennes des dix dernières années. Ils permettent ainsi d'identifier la part des voyageurs concernés par des périodes de forte chaleur et de comparer les conditions climatiques selon le pays sélectionné.

Le graphique en courbe retrace le volume des réservations réparties sur les 12 mois de l'année. Ce visuel met en évidence la saisonnalité marquée de l'activité et permet d'identifier les périodes de pics ou creux de réservations afin d'adapter le calendrier commercial et tarifaire.

L'histogramme groupé compare directement les températures maximales constatées avec les moyennes des dix dernières années. Il permet de repérer les écarts entre les différentes destinations et de mieux adapter les périodes proposées aux conditions climatiques.

La matrice permet de visualiser l'ensemble des températures de l'année pour chaque destination. Son dégradé de couleurs offre une comparaison détaillée en faisant ressortir instantanément les mois les plus chauds de chaque pays. Cet outil permet de connaître les températures selon les saisons afin d'adapter l'offre commerciale aux contraintes météorologiques locales.

4 - Annexes

4.1. Création des tables de la base de données

Toutes les clés primaires de type NUMBER sont supérieures à 0.
Toutes les colonnes des tables sont renseignées sauf si mentionné.
ON DELETE CASCADE permet de supprimer automatiquement les lignes d'une table enfant lorsqu'une ligne référencée de la table parent est supprimée. Cela garantit la cohérence référentielle.

4.1.1. Table CLIENTS

```
CREATE TABLE CLIENTS
(CodeC NUMBER CONSTRAINT pk_clients PRIMARY KEY, -- code d'un client
NomC varchar2 (30) CONSTRAINT nn_clients_nomC NOT NULL, -- nom d'un client
PrenomC varchar2 (30) CONSTRAINT nn_clients_prenomC NOT NULL, -- prénom d'un client
TelC varchar2 (15) CONSTRAINT nn_clients_telC NOT NULL, -- téléphone d'un client
MelC varchar2 (30) CONSTRAINT nn_clients_melC NOT NULL, -- adresse email d'un client
CONSTRAINT ck_clients_codeC CHECK (CodeC>0));
```

4.1.2. Table AEROPORTS_DEP

```
CREATE TABLE AEROPORTS_DEP
( CodeA NUMBER CONSTRAINT pk_aeroports PRIMARY KEY, -- code d'un aéroport
NomA varchar2 (30) CONSTRAINT nn_aeroports_dep_nomA not null, -- nom d'un aéroport
CONSTRAINT ck_aeroports_dep_codeA CHECK (CodeA>0));
```

4.1.3. Table CIRCUITS

```
CREATE TABLE CIRCUITS
( CodeCI NUMBER CONSTRAINT pk_circuits PRIMARY KEY, -- code d'un circuit
CONSTRAINT ck_circuits_codeCI CHECK (CodeCI>0),
NomCI varchar2 (30) constraint nn_circuits_nomCI not null, -- nom d'un circuit
PrixBase NUMBER(6,2) CONSTRAINT nn_circuits_prixBase not null, -- le prix suit un format 0,00€
CONSTRAINT ck_circuits_prixBase check (PrixBase >0), -- prix de base > 0
NbPersMax number CONSTRAINT nn_circuits_NbPersMax not null -- nombre de personnes maximale dans un circuit
CONSTRAINT ck_circuits_NbPersMax check (NbPersMax>0), -- nombre maximum de personnes acceptées par le circuit
```

```
CodeCI_Est_Complete number, -- un circuit ne complète pas forcément un autre circuit donc pas de contraintes NOT NULL
CONSTRAINT fk_CIRCUITCOMPLETE
FOREIGN KEY (CodeCI_Est_Complete) -- code du circuit complétant un autre circuit
REFERENCES CIRCUITS (CodeCI)
ON DELETE CASCADE);
```

4.1.4. Table CHOIX

```
CREATE TABLE CHOIX
( CodeC number, -- code d'un client
CodeA number, -- code d'un aéroport
CodeCI number, -- code d'un circuit
DateDepartC date, -- date de départ du circuit
OrdrePreference number CONSTRAINT nn_choix_ordrePreference not null, -- 1, 2 ou 3
CONSTRAINT ck_OrdrePreference
check (OrdrePreference in (1,2,3)),
CONSTRAINT pk_CHOIX
PRIMARY KEY (CodeC, CodeA, CodeCI, dateDepartC),
CONSTRAINT fk_CHOIXC
FOREIGN KEY ( CodeC)
REFERENCES CLIENTS ( CodeC)
ON DELETE CASCADE ,
CONSTRAINT fk_CHOIXA
FOREIGN KEY (CodeA)
REFERENCES AEROPORTS_DEP (CodeA)
ON DELETE CASCADE,
CONSTRAINT fk_CHOIXCI
FOREIGN KEY (CodeCI)
REFERENCES CIRCUITS (CodeCI)
ON DELETE CASCADE );
```

4.1.5. Table RESERVER

```
CREATE TABLE RESERVER
(CodeC number, -- code d'un client
CodeA number, -- code d'un aéroport
CodeCI number, -- code d'un circuit
DateDepartR date CONSTRAINT nn_reserver_dateDepartR not null, -- date de départ du circuit
DateResa date CONSTRAINT nn_reserver_dateResa not null, -- date de réservation du circuit
NbPers number CONSTRAINT nn_reserver_nbPers not null -- nombre de personnes accompagnant le client pour le circuit
```

```
CONSTRAINT ck_NbPers check (NbPers>0), -- nombre de personnes > 0
CONSTRAINT pk_RESERVER
PRIMARY KEY (CodeC, CodeA, CodeCI, DateDepartR),
CONSTRAINT fk_RESERVERC FOREIGN KEY (CodeC)
REFERENCES CLIENTS ( CodeC) ON DELETE CASCADE,
CONSTRAINT fk_RESERVERA FOREIGN KEY (CodeA)
REFERENCES AEROPORTS_DEP (CodeA) ON DELETE CASCADE,
CONSTRAINT fk_RESERVERCI FOREIGN KEY (CodeCI)
REFERENCES CIRCUITS (CodeCI) ON DELETE CASCADE);
```

4.1.6. Table MAJORATION

```
CREATE TABLE MAJORATION
(CodeA number, -- code d'un aéroport
CodeCI number, -- code d'un circuit
DateDepartM date CONSTRAINT nn_majoration_dateDepartM not null, -- date de départ du circuit
MontantMaj NUMBER(5,2) CONSTRAINT nn_majoration_montantMaj not null -- montant de la majoration en fonction de l'aéroport et la date de départ
CONSTRAINT ck_MontantMaj
check (MontantMaj>0), -- montant de majoration > 0 suivant le format 0,00€
CONSTRAINT pk_MAJORATION
PRIMARY KEY ( CodeA, CodeCI, DateDepartM),
CONSTRAINT fk_MAJORATIONA
FOREIGN KEY (CodeA)
REFERENCES AEROPORTS_DEP (CodeA)
ON DELETE CASCADE,
CONSTRAINT fk_MAJORATIONCI
FOREIGN KEY (CodeCI)
REFERENCES CIRCUITS (CodeCI)
ON DELETE CASCADE );
```

4.1.7. Table HOTELS

```
CREATE TABLE HOTELS
(CodeH NUMBER CONSTRAINT pk_hotels PRIMARY KEY, -- code d'un hotel
CONSTRAINT ck_hotels_codeH CHECK (CodeH>0),
NomH varchar2 (30) CONSTRAINT nn_hotels_nomH not null, -- nom de l'hotel
Categorie number CONSTRAINT nn_hotels_categorie not null -- nombre d'étoiles associé à l'hôtel
CONSTRAINT ck_Categorie check(Categorie>0), -- nombre d'étoiles > 0
VilleH varchar2 (30) CONSTRAINT nn_hotels_villeH not null,
-- le téléphone ou l'adresse email doit être mentionné
TelH varchar2 (20), -- téléphone de l'hotel
MelH varchar2 (20)); -- adresse email de l'hotel
```

4.1.8. Table LOGER

```
CREATE TABLE LOGER
(CodeCI number, -- code d'un circuit
CodeH number, -- code d'un hotel
JourSe varchar2(10) CONSTRAINT nn_loger_jourSe not null, -- jour de la semaine
CONSTRAINT pk_LOGER PRIMARY KEY (CodeCI, CodeH),
CONSTRAINT ck_loger_JourSe
check (JourSe in ('Lundi','Mardi','Mercredi','Jeudi','Vendredi','Samedi','Dimanche')),
CONSTRAINT fk_loger_codeCI FOREIGN KEY (CodeCI)
REFERENCES CIRCUITS (CodeCI) ON DELETE CASCADE,
CONSTRAINT fk_loger_codeH FOREIGN KEY (CodeH)
REFERENCES HOTELS (CodeH) ON DELETE CASCADE);
```

4.1.9. Table ANIMATIONS

```
CREATE TABLE ANIMATIONS (
CodeAN  NUMBER CONSTRAINT pk_animations PRIMARY KEY, -- code d'une animation
NomAN  VARCHAR2(30) CONSTRAINT nn_animations_nomAN NOT NULL, -- nom d'une animation
TypeAN  VARCHAR2(20) CONSTRAINT nn_animations_typeAN NOT NULL, -- type d'une animation
HeureDeb  DATE CONSTRAINT nn_animations_heureDeb NOT NULL, -- heure de début de l'animation
JourSemaine VARCHAR2(10) CONSTRAINT nn_animations_jourSemaine NOT NULL, -- jour de la semaine
CodeH  number CONSTRAINT nn_animations_codeH NOT NULL, -- code de l'hotel qui propose l'animation
CONSTRAINT fk_animations_codeH
FOREIGN KEY (CodeH) REFERENCES HOTELS (CodeH) ON DELETE CASCADE,
CONSTRAINT ck_animations_typeAN
CHECK (TypeAN IN ('Concert', 'Danse folklorique', 'Danse', 'Cabaret', 'Karaoké')), -- énumération des types d'animations disponibles
CONSTRAINT ck_animations_jourSemaine
CHECK (JourSemaine IN ('Lundi', 'Mardi', 'Mercredi', 'Jeudi', 'Vendredi', 'Samedi', 'Dimanche')),
CONSTRAINT ck_animations_codeAN CHECK (CodeAN > 0)
);
```

4.1.10. Table ACTIVITE

```
CREATE TABLE ACTIVITE
(CodeAC NUMBER CONSTRAINT pk_activites PRIMARY KEY, -- code d'une activité
```

```
NomAC varchar2(30) CONSTRAINT nn_activite_nomAC not null, -- nom de l'activité
TarifAC number(6,2) CONSTRAINT nn_activite_tarifAC not null, -- valeur monétaire au format 0,00€
CONSTRAINT ck_activite_tarifAC check (TarifAC >0), -- tarif de l'activité > 0
CONSTRAINT ck_activite_codeAC check (codeAC>0));
```

4.1.11. Table VALIDITE

```
CREATE TABLE VALIDITE (
 -- code d'une activité
 CodeAC number,
 -- date de début de validité d'une activité
 DateDebut date,
 -- date de fin de validité d'une activité
 DateFin date CONSTRAINT nn_validite_dateFin not null,
 -- clé primaire : une activité a une seule période par date de début
 CONSTRAINT pk_validite PRIMARY KEY (CodeAC, DateDebut),
 -- l'activité doit exister
 CONSTRAINT fk_validite_codeAC FOREIGN KEY (CodeAC)
   REFERENCES ACTIVITE (CodeAC) ON DELETE CASCADE,
 -- la date de fin est postérieure ou égale à la date de début
 CONSTRAINT ck_validite_dates CHECK (DateFin >= DateDebut));
```

4.1.12. Table REALISER

```
CREATE TABLE REALISER (
  CodeCI       number, -- code d'un circuit
  CodeAC       number, -- code d'une activité
  DateDepartRe  DATE, -- date de départ du circuit
   JourSem       VARCHAR2(10) CONSTRAINT nn_realiser_jourSem NOT NULL, -- jour de la semaine associé à la réalisation d'une activité pour un circuit
  CONSTRAINT pk_realiser PRIMARY KEY (CodeCI, CodeAC, DateDepartRe),
  CONSTRAINT fk_realiser_codeCI FOREIGN KEY (CodeCI)
     REFERENCES CIRCUITS (CodeCI) ON DELETE CASCADE,
  CONSTRAINT fk_realiser_codeAC FOREIGN KEY (CodeAC)
     REFERENCES ACTIVITE (CodeAC) ON DELETE CASCADE,
  CONSTRAINT ck_realiser_jourSem CHECK (JourSem IN
     ('Lundi', 'Mardi', 'Mercredi', 'Jeudi',
      'Vendredi', 'Samedi', 'Dimanche'))
);
```

4.1.13. Table SITES

```
CREATE TABLE SITES (
```

```
    CodeS    number CONSTRAINT pk_sites PRIMARY KEY, -- code du site touristique
   NomS        VARCHAR2(30)  CONSTRAINT nn_sites_nomS NOT NULL, -- nom d'un site touristique
   TypeS       VARCHAR2(20)  CONSTRAINT nn_sites_typeS NOT NULL, -- type d'un site touristique
    DescripS     VARCHAR2(400) CONSTRAINT nn_sites_DescripS NOT NULL, -- description d'un site touristique
  CONSTRAINT ck_sites CHECK (CodeS > 0),
  CONSTRAINT ck_sites_typeS CHECK (TypeS IN
      ('historique', 'religieux', 'paysage', 'commerce', 'autre')) -- énumération des types de sites touristiques disponibles
);
```

4.1.14. Table VISITER

```
CREATE TABLE VISITER (
  CodeCI      number, -- code du circuit
  CodeS       number, -- code du site touristique
   JourSe2       VARCHAR2(10) CONSTRAINT nn_visiter_JourSe2 NOT NULL, -- jour de la semaine
  HeureVisite  DATE CONSTRAINT nn_visiter_heureVisite NOT NULL, -- HH:MM
    DureeVisite   number CONSTRAINT nn_visiter_dureeVisite NOT NULL, -- en minutes
  CONSTRAINT pk_visiter PRIMARY KEY (CodeCI, CodeS, JourSe2, HeureVisite),
  CONSTRAINT fk_visiter_codeCI FOREIGN KEY (CodeCI)
     REFERENCES CIRCUITS (CodeCI) ON DELETE CASCADE,
  CONSTRAINT fk_visiter_codeS FOREIGN KEY (CodeS)
     REFERENCES SITES (CodeS) ON DELETE CASCADE,
  CONSTRAINT ck_visiter_jourSe2 CHECK (JourSe2 IN
     ('Lundi', 'Mardi', 'Mercredi', 'Jeudi',
      'Vendredi', 'Samedi', 'Dimanche')),
  CONSTRAINT ck_visiter_dureeVisite CHECK (DureeVisite > 0) -- durée de visite > 0
);
```

4.1.15. Table MOIS

```
CREATE TABLE MOIS (
  CodeMois    NUMBER CONSTRAINT pk_mois PRIMARY KEY, -- code du mois
   LibelleMois VARCHAR2(20) CONSTRAINT nn_mois_libelleMois NOT NULL, -- nom du mois
   CONSTRAINT ck_mois_codeMois    CHECK (CodeMois BETWEEN 1 AND 12), -- codeMois = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12 }
  CONSTRAINT ck_mois_libelleMois CHECK (LibelleMois IN
     ('Janvier', 'Février', 'Mars', 'Avril', 'Mai', 'Juin', 'Juillet',
```

```
        'Août', 'Septembre', 'Octobre', 'Novembre', 'Décembre'))
);
```

4.1.16. Table PAYS

```
CREATE TABLE PAYS (
  CodeP      NUMBER CONSTRAINT pk_pays PRIMARY KEY, -- code du pays
  NomP       VARCHAR2(30) CONSTRAINT nn_pays_nomP NOT NULL, -- nom du pays
  AvisGen    VARCHAR2(40) CONSTRAINT nn_pays_avisGen NOT NULL, -- avis général du pays
  CONSTRAINT ck_pays_codeP CHECK (CodeP>0)
);
```

4.1.17. Table AVIS

```
CREATE TABLE AVIS (
  CodeP      number, -- code du pays
  CodeMois   number, -- code du mois
    TempMin        NUMBER(5,2) CONSTRAINT nn_avis_tempMin NOT NULL, -- température minimale d'un pays pour un mois donné
    TempMax        NUMBER(5,2) CONSTRAINT nn_avis_tempMax NOT NULL, -- température maximale d'un pays pour un mois donné
  AvisMens   VARCHAR2(40) CONSTRAINT nn_avis_avisMens NOT NULL, -- avis mensuel de la température d'un pays
  TempMinMoy NUMBER(5,2) CONSTRAINT nn_avis_tempMinMoy NOT NULL, -- température minimale moyenne sur les 10 dernières années
  TempMaxMoy NUMBER(5,2) CONSTRAINT nn_avis_tempMaxMoy NOT NULL, -- température maximale moyenne sur les 10 dernières années
  CONSTRAINT pk_avis PRIMARY KEY (CodeP, CodeMois),
  CONSTRAINT fk_avis_codeP FOREIGN KEY (CodeP)
    REFERENCES PAYS (CodeP) ON DELETE CASCADE,
  CONSTRAINT fk_avis_codeMois FOREIGN KEY (CodeMois)
    REFERENCES MOIS (CodeMois) ON DELETE CASCADE,
  CONSTRAINT ck_avis_temp   CHECK (TempMax >= TempMin), -- la température maximale d'un pays est supérieure ou égale à sa température minimale
  CONSTRAINT ck_avis_tempmoy CHECK (TempMaxMoy >= TempMinMoy) -- la température maximale moyenne d'un pays sur les 10 dernières années est
    -- supérieure ou égale à la température minimale du pays sur les 10 dernières années
);
```

4.1.18. Table Zone_Geo

```
CREATE TABLE Zone_Geo (
  CodeZ      NUMBER CONSTRAINT pk_zone_geo PRIMARY KEY, -- code d'une zone géographique
```

```
  ZoneGeo     VARCHAR2(15) NOT NULL, -- nom de la zone géographique
  CONSTRAINT ck_zone_geo_codeZ CHECK (CodeZ > 0),
  CONSTRAINT ck_zone_geo_zoneGeo CHECK (ZoneGeo IN
     ('Nord', 'Sud', 'Ouest', 'Est',
      'Nord-Ouest', 'Nord-Est', 'Sud-Est', 'Sud-Ouest')) -- valeurs prises par le nom de
la zone géographique
);
```

4.1.19. Table SITUER

```
CREATE TABLE SITUER (
  CodeCI      number, -- code du circuit
  CodeZ       number, -- code de la zone géographique
  CodeP       number, -- code du pays
  CONSTRAINT pk_situer PRIMARY KEY (CodeCI, CodeZ, CodeP),
  CONSTRAINT fk_situer_codeCI FOREIGN KEY (CodeCI)
     REFERENCES CIRCUITS (CodeCI) ON DELETE CASCADE,
  CONSTRAINT fk_situer_codeZ FOREIGN KEY (CodeZ)
     REFERENCES Zone_Geo (CodeZ) ON DELETE CASCADE,
  CONSTRAINT fk_situer_codeP FOREIGN KEY (CodeP)
     REFERENCES PAYS (CodeP) ON DELETE CASCADE
);
```

4.1.20. Table GUIDES

```
CREATE TABLE GUIDES (
  CodeG       NUMBER CONSTRAINT pk_guides PRIMARY KEY, -- code du guide
  NomG        VARCHAR2(30) CONSTRAINT nn_guides_nomG NOT NULL, -- nom
du guide
   PrenomG      VARCHAR2(30) CONSTRAINT nn_guides_prenomG NOT NULL, --
prenom du guide
   DatePremier   DATE  CONSTRAINT  nn_guides_datePremier  NOT  NULL, -- date
d'entrée du guide au sein de la société
  CONSTRAINT ck_guides_codeG CHECK (codeG > 0)
);
```

4.1.21. Table ACCOMPAGNER

```
CREATE TABLE ACCOMPAGNER (
  CodeG       number, -- code du guide
  CodeCI      number, -- code du circuit
  DateDepartA  DATE    CONSTRAINT nn_accompagner_dateDepart NOT NULL, --
date de départ du circuit
  CONSTRAINT pk_accompagner PRIMARY KEY (CodeG, CodeCI, DateDepartA),
  CONSTRAINT fk_accompagner_codeG FOREIGN KEY (CodeG)
     REFERENCES GUIDES (CodeG) ON DELETE CASCADE,
  CONSTRAINT fk_accompagner_codeCI FOREIGN KEY (CodeCI)
```

```
    REFERENCES CIRCUITS (CodeCI) ON DELETE CASCADE
);
```

4.1.22. Table LANGUE

```
CREATE TABLE LANGUE (
   CodeL       NUMBER CONSTRAINT pk_langue PRIMARY KEY, -- code de la langue
  NomL       VARCHAR2(40) CONSTRAINT nn_langue_nomL NOT NULL, -- nom de la langue
 CONSTRAINT nn_langue_codeL CHECK (CodeL > 0)
);
```

4.1.23. Table PRATIQUER

```
CREATE TABLE PRATIQUER (
  CodeG      number, -- code du guide
  CodeL      number, -- code de la langue
  NiveauL      VARCHAR2(20) CONSTRAINT nn_pratiquer_niveauL NOT NULL, -- niveau de langue du guide pour une langue donnée
  CONSTRAINT pk_pratiquer PRIMARY KEY (CodeG, CodeL),
  CONSTRAINT fk_pratiquer_codeG FOREIGN KEY (CodeG)
    REFERENCES GUIDES (CodeG) ON DELETE CASCADE,
  CONSTRAINT fk_pratiquer_codeL FOREIGN KEY (CodeL)
    REFERENCES LANGUE (CodeL) ON DELETE CASCADE,
  CONSTRAINT ck_pratiquer_niveauL CHECK (NiveauL IN
      ('excellent', 'bon', 'moyen', 'quelques notions', 'mots de base')) -- valeurs prises par le niveau de langue
);
```

4.2. Création de la vue ANIMATIONS_H

Nous avons créé la vue ANIMATIONS_H afin d'afficher l'heure de début des animations au format HH24:MI. La colonne HeureDeb étant de type DATE sous Oracle, la fonction TO_CHAR permet d'afficher uniquement l'heure et les minutes, sans modifier les données de la table d'origine.

```
-- Vue de la table animation permettant d’afficher heuredeb au format hh:mm
CREATE VIEW ANIMATIONS_H as
SELECT CodeAN, NomAN, TypeAN, TO_CHAR(HEUREDEB, 'HH24:MI') as HeureDEB, JourSemaine, CodeH
FROM ANIMATIONS;
```

4.3 Création du trigger sur RESERVER

Le dictionnaire de données définit NbPersMax comme le nombre maximum de voyageurs acceptés pour un circuit.

Cette contrainte ne peut pas être vérifiée avec un simple CHECK, car elle nécessite de comparer les données des tables RESERVER et CIRCUITS.

Nous avons donc créé un trigger permettant de contrôler le nombre total de voyageurs inscrits pour un même circuit et une même date de départ, tous aéroports confondus.

Code SQL :

```
-- Vérifie que le nombre total de voyageurs
-- ne dépasse pas la capacité maximale du circuit
CREATE OR REPLACE TRIGGER trg_reserver_nbpers
AFTER INSERT OR UPDATE ON RESERVER

DECLARE
   -- Nombre de départs dépassant leur capacité
   v_depasse NUMBER;

BEGIN
   -- Recherche les départs dont le total de voyageurs
   -- dépasse la capacité maximale autorisée
   SELECT COUNT(*) INTO v_depasse
   FROM (
      SELECT r.CodeCI, r.DateDepartR
      FROM RESERVER r, CIRCUITS c
      WHERE r.CodeCI = c.CodeCI

      -- Regroupe les réservations par circuit et date,
      -- tous aéroports confondus
      GROUP BY r.CodeCI, r.DateDepartR, c.NbPersMax

      -- Compare les voyageurs inscrits
      -- à la capacité maximale du circuit
      HAVING SUM(r.NbPers) > c.NbPersMax
   );

   -- Si un départ dépasse sa capacité,
   -- une erreur empêche la réservation
```

```
    IF v_depasse > 0 THEN
        RAISE_APPLICATION_ERROR(-20001,
            'Capacité maximale du circuit dépassée');
    END IF;

END;
/
```

Justification :

Le trigger se déclenche après chaque instruction d'insertion ou de modification dans la table RESERVER. Il utilise GROUP BY pour regrouper les réservations par circuit et date de départ, puis HAVING pour identifier les départs dont le nombre total de voyageurs dépasse NbPersMax.

Si au moins un dépassement est détecté, RAISE_APPLICATION_ERROR génère une erreur et annule l'opération concernée.

Nous avons choisi un trigger AFTER sans FOR EACH ROW afin de pouvoir consulter la table RESERVER après sa modification, sans provoquer l'erreur Oracle de table mutante (ORA-04091).

Limite :

Cette solution vérifie le cumul des voyageurs après chaque instruction, mais ne garantit pas entièrement la gestion des réservations effectuées simultanément par plusieurs transactions.